

Product Agent 答辩问答清单

使用建议：这份文档不是逐字稿，而是答辩“弹药库”。正式答辩只有几分钟时，优先记住每类问题的核心口径；老师追问时再展开细节。

一句话定位

Product Agent 是一个面向科研调研早期阶段的证据驱动型研究助手。它不是只回答“这个方向是什么”，而是把论文检索、PDF 导入、证据池、研究方向图、论文关系图、多轮追问和研究记忆组织成一个闭环工作台，帮助用户判断“该读什么、证据在哪里、这个方向还缺什么、下一步怎么做”。

30 秒产品介绍口径

传统科研调研的问题不是完全没有答案，而是信息分散、论文很多、AI 给出的建议缺少可追溯依据。Product Agent 的目标是把“找论文、读论文、整理方向、提出后续问题”放进同一个工作流里。用户输入一个研究主题，可以选择上传自己的 PDF、让系统自动检索论文，或者两者结合。系统会构建论文证据池，生成研究简报、论文证据、研究结构图、知识上下文和运行轨迹，并支持后续自然语言追问。每轮追问都会尽量继承已有研究上下文，并在需要时补搜新论文，而不是只进行普通聊天。

一、产品背景与价值

Q1：你们为什么做这个产品？

推荐回答：

科研调研早期经常有三个痛点：第一，搜索结果很多，但不知道哪些论文真正值得先读；第二，AI 可以直接给建议，但用户很难判断依据来自哪里；第三，多轮调研过程中，上一次的论文、空白和结论很容易散掉。我们做 Product Agent，就是希望把论文证据和 AI 生成结果绑定起来，让调研过程从“问一个 AI”变成“持续维护一个有证据的研究工作台”。

Q2：用户为什么不用普通 ChatGPT 或普通搜索引擎？

推荐回答：

普通 AI 更像一次性回答，普通搜索引擎更像论文列表。Product Agent 的区别是把两者连接起来：先形成 evidence pool，再生成研究结构、研究空白和建议，并在前端展示每个结论依赖哪些论文、哪些是用户上传、哪些是系统检索、哪些证据强、哪些还需要验证。也就是说，我们不是只生成答案，而是帮用户组织“可复盘的调研过程”。

Q3：你们的产品化亮点是什么？

推荐回答：

主要有四点。第一，支持用户上传 PDF 与系统检索结合，适合用户已有几篇种子论文的场景。第二，支持自然语言追问，追问会转成新的研究任务，并继承已有论文池和研究记忆。第三，结果不是单段文字，而是拆成研究简报、核心论文、研究空白、研究建议、taxonomy 和演进图。第四，我们在前端展示运行轨迹和证据来源，降低黑箱感。

Q4：产品主要面向什么使用场景？

推荐回答：

最适合的是课程项目选题、文献综述早期、开题前方向探索、已有几篇论文但还没形成结构化理解的场景。比如学生已经下载了 3-6 篇论文，但不知道这个方向有哪些分支、该先读哪几篇、还有哪些空白可以继续做，Product Agent 可以帮助建立第一版研究地图。

二、整体架构与系统构成

Q5：系统整体架构是什么？

推荐回答：

系统采用前后端分离。前端是 React/Vite，用工作台形式展示对话、研究总览、论文证据、研究结构和运行轨迹；后端是 FastAPI，负责会话、任务、论文检索、PDF 解析、知识库、Agent workflow 和结果生成；数据层使用 SQLite 做本地持久化，包括对话、消息、研究任务、工作台快照、知识文档、向量 chunk、论文池和运行事件。

Q6：系统分为几个主要模块？

推荐回答：

可以分成七类模块：

1. **交互层**：React 前端，提供新建研究、上传 PDF、追问、工作台和设置页。
2. **API 层**：FastAPI，对外暴露会话、消息、研究任务、知识库、论文导入、工具和运行配置接口。
3. **Agent 工作流层**：由 Controller 调度 Planner、Searcher、Taxonomy、Evolution、Auditor、Corrector、Synthesizer 等节点。
4. **论文与证据层**：负责外部论文检索、元数据补全、PDF 解析、论文池管理和证据准入。
5. **知识与上下文层**：包括共享知识库、当前研究资料、working memory、历史工作台摘要和 RAG 检索。
6. **结果生成层**：生成研究简报、研究空白、研究建议、论文详情、taxonomy 和研究关系图。
7. **工程支撑层**：SQLite 持久化、任务事件、运行状态可视化、测试集、CI 和配置管理。

Q7：为什么不用 LangChain 这类框架？

推荐回答：

我们目前没有直接使用 LangChain，而是实现了轻量的 agent workflow。原因是课程项目更强调

可解释、可控和可答辩。我们希望清楚知道每一步什么时候检索、什么时候审计、什么时候补搜、什么时候生成结论，而不是把核心逻辑藏在大框架里。当然，如果后续做更大规模产品，可以考虑把部分工具编排、记忆管理或 tracing 接入成熟框架。

Q8：系统使用了哪些外部能力？

推荐回答：

主要包括 LLM API、arXiv 检索、Semantic Scholar/OpenAlex 等论文元数据来源、PDF 解析工具、本地 SQLite 存储和前端可视化。LLM 主要用于查询理解、结构化生成、研究建议、研究简报压缩等；论文检索和元数据来源用于构建证据池；PDF 解析用于把用户上传论文变成可检索知识。

三、上传论文与知识库

Q9：用户上传的 PDF 是怎么处理的？

推荐回答：

用户上传 PDF 后，前端会通过 multipart form-data 调用后端导入接口。后端的 PdfImportService 会做几件事：先校验文件类型、大小和 PDF 头；再用 PDF parser 提取页级文本；然后推断论文标题，并尝试做外部元数据匹配；最后把全文按 page-aware 的形式保存成 KnowledgeDocument，同时把论文加入当前研究的 ResearchPaper 列表。这样上传论文既能作为知识 chunk 被检索，也能作为论文证据参与后续分析。

Q10：上传论文会参与哪些流程？

推荐回答：

上传论文不是只显示在列表里，它会参与多个流程：

1. **当前研究证据池**：作为 user_upload 论文进入 paper pool。
2. **RAG 上下文**：PDF 正文会被切成 chunk，用于后续问题和追问的知识命中。
3. **核心论文选择**：如果与主题相关，会和系统检索论文一起竞争核心论文位置，并且有用户上传权重。
4. **taxonomy 构建**：论文标题、摘要、正文片段和分类信息会影响研究方向图。
5. **演进图构建**：论文会作为节点参与关系图，可能形成引用、互补、主题关联等边。
6. **研究空白和建议**：上传论文可以作为某个研究方向已有证据，也可能暴露某个方向证据不足。
7. **论文详情**：前端会显示为什么建议读、适合支撑什么、证据级别和进一步验证建议。

Q11：全局共享知识和当前研究上传论文有什么区别？

推荐回答：

当前研究上传论文属于某一次对话或研究，它主要服务于当前主题；全局共享知识是跨对话可复

用的背景知识。系统在检索知识时会根据 `knowledge_scope` 控制范围，既可以只用当前研究资料，也可以同时使用全局共享知识。这样做是为了避免无关历史资料污染新主题。

Q12：历史对话的论文会自动进入共享知识吗？

推荐回答：

不会无脑进入全局共享知识。历史对话的论文和结论主要沉淀在当前研究总览和 `working memory` 中；共享知识更偏显式导入。这样可以防止旧对话里跑偏的内容影响新研究。我们认为这是产品可信性的一个重要边界。

Q13：如果老师问“你们是不是做了 RAG”，怎么回答？

推荐回答：

我们做了轻量 RAG。具体来说，知识文档会被切成 `chunk` 并建立向量/关键词检索索引；生成研究上下文时，系统会从当前研究资料和共享知识库中取相关命中，附带 `scope`、`source_type`、`score` 和 `evidence_level`。它不是一个大规模生产级 RAG 平台，但已经具备文档导入、`chunk` 检索、作用域控制和证据展示这些核心能力。

四、Agent 工作流与节点职责

Q14：你们的 agent 是怎么工作的？

推荐回答：

我们实现的是一个 Controller 驱动的 agent workflow。Controller 根据当前状态决定下一步做什么。如果没有检索计划，就进入 Planner；没有论文证据，就进入 Searcher；taxonomy 缺失就进入 Taxonomy；关系图缺失就进入 Evolution；生成前先进入 Auditor 和 Corrector；最后由 Synthesizer 生成用户可读结果。这个流程不是固定直线，而是可以因为审计发现问题而回到 Searcher 补搜。

Q15：Agent 有哪些 node？分别负责什么？

推荐回答：

核心 action node 有七个，加上 Controller 和前端展示用的 Result 状态：

1. **Controller**：调度中枢，根据状态决定下一个节点。
2. **Planner**：理解用户主题和追问，生成检索计划、子问题和查询组合。
3. **Searcher**：调用 arXiv、Semantic Scholar 等工具，执行多源检索、补搜和相关性过滤。
4. **Taxonomy**：根据论文证据构建研究方向分类，也就是研究方向图的基础。
5. **Evolution**：构建论文之间的关系，包括引用、改进、互补、评测范围扩展、主题关联等。
6. **Auditor**：审计当前结果质量，检查 taxonomy 覆盖、图关系质量和证据是否足够。
7. **Corrector**：根据审计结果决定是否需要补搜、刷新 taxonomy 或降级生成。
8. **Synthesizer**：把内部状态转换成研究简报、研究空白、研究建议和可视化结果。
9. **Result**：前端状态图中的终点，表示当前轮结果已经可以展示。

Q16：为什么需要 Auditor 和 Corrector？

推荐回答：

如果没有 Auditor 和 Corrector，系统可能搜到几篇论文就直接生成结论，容易出现论文少、跑题或证据不足的问题。Auditor 负责检查“现在的证据够不够、结构是否合理、图关系是否可信”；Corrector 负责根据审计结果决定是否补搜或保守降级。它们是抑制幻觉和形成闭环的关键。

Q17：追问时会重新跑完整流程吗？

推荐回答：

追问会被转成新的 follow-up research task，不是普通聊天补一句。系统会读取已有工作台、论文池、working memory 和用户追问，再判断是否需要补搜。如果用户要求“近三年”“换方向”“补充某类论文”，Searcher 会尝试按新约束检索，而不是只在旧论文里挑。

Q18：什么情况下会补搜？

推荐回答：

几类情况会触发补搜：用户追问改变范围或时间约束；当前证据池覆盖不足；Auditor 发现 taxonomy 分支缺直接论文；Corrector 判断需要修复证据空白；已有论文与新问题语义匹配不足。补搜的目标是让研究随着追问真的推进，而不是只复述旧结果。

五、检索、证据与可信性

Q19：你们如何确保搜到的论文相关？

推荐回答：

我们做了几层控制。Planner 会先把自然语言主题拆成检索意图、时间范围、关注维度和查询组合；Searcher 会多源召回论文；召回后会进行相关性评分和过滤；如果严格查询结果不足，会做扩展检索或 facet rescue；最后在 evidence pool 中区分核心分析论文和扩展证据，避免一开始搜到的弱相关论文直接污染结论。

Q20：如果搜不到论文怎么办？

推荐回答：

系统会先尝试查询扩展、多源召回、补搜和降级。如果证据仍不足，会把结果标记为证据受限，并建议用户上传论文、缩小主题或改变查询方向。我们宁愿告诉用户“当前证据不足”，也不希望拿无关论文强行生成看似完整的结论。

Q21：结果为什么可信？

推荐回答：

我们不是要求用户无条件相信模型，而是提供可回溯依据。每个研究结果背后都有论文池、核心论文、知识命中、证据等级和运行轨迹。研究空白和建议也会尽量绑定论文或 taxonomy 分支。系统把“结论从哪里来”展示出来，让用户能复盘和质疑。

Q22：如何抑制 AI 幻觉？

推荐回答：

主要靠四层机制：

1. **证据优先**：先构建论文池和知识命中，再生成结果。
2. **结论准入**：没有论文支持的内容不能直接当强结论，只能作为待验证假设或研究空白。
3. **审计修正**：Auditor 和 Corrector 会检查覆盖不足、关系弱和跑题问题。
4. **前端暴露证据**：用户能看到论文来源、是否用户上传、引用、分类、证据等级和上下文中。

Q23：研究空白和研究建议是怎么得出来的？

推荐回答：

研究空白来自 taxonomy 覆盖检查、审计结果和证据池分布。比如某个方向有概念或需求，但当前论文支持少，就会被标成空白。研究建议则在空白和核心论文基础上生成，要求绑定论文证据或明确标成探索性方向。我们后续也加入了更保守的准入逻辑，避免凭空给“高大上”的建议。

Q24：演进图的边是怎么来的？

推荐回答：

演进图不是随便连线。系统会综合论文引用、标题摘要关键词、taxonomy 分支、方法或评测对象关系，推断几类边：引用关系、改进关系、互补视角、评测范围扩展、主题关联等。对于强证据关系会提高置信度；普通主题关联只作为候选线索，不包装成确定的学术演进结论。

Q25：论文详情为什么有“摘要级”“全文级”之分？

推荐回答：

因为很多论文系统最初只能拿到标题、摘要和元数据，不能直接保证全文 claim 都被验证。我们在论文详情里区分摘要级判断、本地 PDF 正文片段核查和更完整的 claim verification。这样做是为了诚实表达证据强度，而不是把摘要判断伪装成全文阅读结论。

六、上下文管理与多轮闭环

Q26：你们的上下文管理具体是什么？

推荐回答：

上下文分为五层：第一是用户和 assistant 的消息；第二是当前研究任务和工作台快照；第三是当前研究的论文证据池；第四是 working memory，记录多轮研究中的稳定结论、开放问题和约束；第五是全局共享知识库。系统在每轮生成前会按范围选择这些上下文，而不是把所有历史内容一股脑塞给模型。

Q27：working memory 记录什么？

推荐回答：

working memory 记录的是当前研究主题下比较稳定的信息，比如研究主题、约束、已确认的重点方向、关键发现、开放问题、上轮追问留下的下一步线索。它的作用是让多轮追问不是孤立的，让系统知道用户已经研究到哪里。

Q28：研究总览如何继承追问内容？

推荐回答：

研究总览不是单次任务结果，而是把同一个 conversation 下多个已完成任务的工作台进行合并。合并时会处理论文池、taxonomy、研究空白、建议、alignment score 和追问演化信息。近期追问会有一定权重，但不会完全丢掉早期高价值论文。

Q29：整体闭环是什么？

推荐回答：

闭环可以概括为：用户提出研究主题，系统规划和检索论文；论文进入证据池；系统生成方向图、演进图、空白和建议；用户基于结果继续追问；追问触发新的检索和审计；新结果合并回研究总览和 working memory。这样用户不是一轮轮散乱提问，而是在同一个研究对象上持续推进。

七、工程优化与实现质量

Q30：你们做了哪些工程上的优化？

推荐回答：

主要有八类：

1. **模块化服务拆分**：把会话、消息、研究任务、知识库、PDF 导入、论文池、工作台、working memory 分成独立 service。
2. **持久化与可复盘**：用 SQLite 保存任务、消息、workspace、知识文档、论文和 task event。
3. **查询分解与覆盖检查**：把自然语言主题拆成检索意图、关注维度和时间范围，提高对随意输入的适配。
4. **多源召回与相关性过滤**：结合 arXiv、Semantic Scholar/OpenAlex，并对弱相关论文做过滤。
5. **并行检索优化**：Searcher 支持并行执行多条查询和多源检索，降低用户等待时间。
6. **证据准入和降级策略**：证据不足时抑制研究建议或标成待验证，避免强行生成。
7. **运行状态可视化**：把 planner/searcher/auditor 等节点事件展示给用户，减少黑箱等待感。
8. **测试与 CI**：针对查询分解、检索补救、论文相关性、知识召回等关键逻辑建立测试。

Q31：为什么要做并行化？

推荐回答：

论文检索天然就是 IO 密集型任务，多条查询、多个数据源可以并发执行。我们在 Searcher 中使用

并行任务，把 arXiv 和 Semantic Scholar 等召回同时跑，减少等待时间。它不是为了炫技，而是因为用户在前端等待研究结果时，检索速度直接影响产品体验。

Q32：系统运行模式有什么区别？

推荐回答：

运行模式主要是在速度和覆盖之间取舍。快速模式更保守，查询数量和外部调用更少，适合演示或快速预览；均衡模式会保留更多查询和多源召回，适合质量和速度都要考虑的主链；更完整的模式可以更重视覆盖和补搜，但等待时间会更长。

Q33：你们怎么保证不是堆功能？

推荐回答：

我们的主线不是“加很多按钮”，而是围绕科研调研闭环设计。每个功能都对应一个问题：PDF 导入解决用户已有资料无法利用；证据池解决结论不可追溯；taxonomy 解决方向结构不清；演进图解决论文关系不清；working memory 解决多轮追问割裂；运行图解决等待黑箱。功能之间是有主线的。

八、前端与交互设计

Q34：为什么工作台要拆成多个区域？

推荐回答：

科研结果信息密度很高，如果只用聊天气泡，很难承载论文、图、证据和建议。所以我们把工作台拆成 Overview、Evidence、Map、Context、Run 等区域。Overview 看结论，Evidence 看论文，Map 看结构，Context 看知识命中，Run 看系统运行过程。

Q35：Agent 运行图有什么意义？

推荐回答：

生成研究结果不是瞬间完成的，中间有规划、检索、审计、修正和生成。运行图能让用户知道系统正在做哪一步、有没有补搜、有没有降级，也方便答辩时展示“我们的 agent 不是一个黑箱 prompt，而是有明确工作流的”。

Q36：为什么要显示论文来源和用户上传标记？

推荐回答：

因为用户需要知道证据来自哪里。用户上传论文通常代表用户自己信任或希望重点分析的材料；系统检索论文代表外部补充证据。把来源标清楚后，用户才能判断结果是在利用自己的资料，还是系统从公开论文库补充出来的。

九、演示相关问题

Q37：演示时为什么要提前导入共享知识？

推荐回答：

真实用户常常不是空手开始调研，而是已经有几篇种子论文。提前导入共享知识是为了模拟“用户已有资料库”的场景。这样系统既能从外部检索新论文，也能利用用户积累的论文，体现产品的持续沉淀价值。

Q38：如果演示时搜到奇怪论文怎么办？

推荐回答：

这是开放检索系统会遇到的真实问题。我们的策略是尽量通过主题约束、查询分解、相关性过滤和证据准入降低跑偏。如果仍然出现不相关论文，系统应当把它们过滤或放入扩展证据，而不是让它们主导结论。演示时也会选择边界清晰的主题，比如可信 RAG 评测、检索失败检测和幻觉缓解。

Q39：推荐演示顺序是什么？

推荐回答：

建议顺序是：先讲痛点，再展示新建研究和上传 PDF，再运行一次主链，看 Overview 和 Evidence；然后展示 Map 里的研究结构和论文关系；再打开 Context 说明共享知识命中；最后发起一次自然语言追问，展示系统如何继承上下文、补搜和更新研究总览。

Q40：如果老师问“你们系统现在有什么不足”，怎么回答？

推荐回答：

可以坦诚说三点。第一，全文级 claim verification 还可以继续增强，目前不是每篇论文都能做到完整全文事实核验。第二，全局 PDF 上传入口还可以更产品化，目前主要能力已经在后端和当前研究上传中体现。第三，演进图关系仍是辅助研究线索，不应被解释成严格的学术因果链。我们后续会围绕更强全文验证、更细粒度论文阅读和更稳定的多源检索继续迭代。

十、较难追问的推荐回答

Q41：你们如何防止共享知识污染新研究？

推荐回答：

我们用 knowledge_scope 做作用域控制。当前研究资料、全局共享知识和历史对话不是混在一起无差别使用。共享知识是显式导入的，当前研究文档有 conversation_id，系统检索时会区分 scope，避免其他对话的私有资料泄漏到新研究中。

Q42：核心论文是怎么选出来的？

推荐回答：

核心论文不是简单按引用数排序，而是综合相关性、是否直接匹配主题、是否来自用户上传、年份、分类覆盖、引用信息和证据作用。高引用论文有优势，但不会压过主题相关性；用户上传论文如果与主题相关，也会得到更高优先级。

Q43：如果 LLM 生成的建议看起来合理但没有论文支持怎么办？

推荐回答：

这正是我们做结论准入的原因。没有论文支持的建议不能直接作为强结论，只能作为探索性方向或待验证假设；如果证据太弱，系统会抑制建议生成。我们的目标不是让 AI 永远给满答案，而是让它在证据不足时学会克制。

Q44：你们怎么定义“产品化”？

推荐回答：

我们理解的产品化不是把算法 demo 包一层 UI，而是让普通用户能完成一条真实任务链。对 Product Agent 来说，就是从“我有一个研究主题和几篇论文”开始，到“我知道该读哪几篇、有哪些方向、证据在哪里、下一步怎么追问”结束。我们围绕这条链路做了工作台、状态反馈、PDF 导入、共享知识、多轮追问和研究总览。

Q45：你们这个系统更像 agent 还是 RAG？

推荐回答：

它是 agent workflow 和 RAG 的结合。RAG 负责让系统能引用用户上传和共享知识；agent workflow 负责决定什么时候检索、什么时候审计、什么时候补搜、什么时候生成结果。单纯 RAG 解决“从文档里找答案”，而我们的 agent 更关注“如何完成一轮科研调研任务”。

十一、答辩表达技巧

不建议这样说

- “我们用了 AI 自动生成研究建议，所以很智能。”
- “演进图能准确表示论文之间的真实发展关系。”
- “上传论文后系统就完全读懂全文了。”
- “我们的系统可以替代人工文献综述。”

建议这样说

- “我们把 AI 生成约束在论文证据池之上，让用户能追溯依据。”
- “演进图是辅助理解研究脉络的候选关系图，不替代人工判断。”
- “上传 PDF 后系统能解析正文并参与检索、证据池和论文详情，但全文级 claim verification 仍是后续增强方向。”
- “系统面向文献调研早期，帮助用户快速建立第一版研究地图。”

十二、最值得背下来的五句话

1. Product Agent 的价值不是替用户编答案，而是把论文证据、研究结构和多轮追问沉淀到同一个工作台。

2. 用户上传 PDF 会被解析成全文知识 chunk，同时作为当前研究论文证据参与 taxonomy、演进图、空白和建议生成。
3. 我们的 agent workflow 由 Controller 调度 Planner、Searcher、Taxonomy、Evolution、Auditor、Corrector 和 Synthesizer，支持审计后补搜。
4. 为了降低幻觉，我们做了证据优先、结论准入、审计修正和前端证据回溯。
5. 我们不把系统定位成替代研究者，而是帮助研究者更快完成“该读什么、证据在哪、下一步做什么”的早期调研闭环。

十三、补充高频追问

Q46：研究空白到底是怎么得来的？

推荐回答：

研究空白不是让 LLM 随便想几个方向。系统会先基于论文池构建 taxonomy，也就是当前证据覆盖到的研究分支；然后结合每个分支下的论文数量、直接证据数量、关键概念覆盖和审计结果，判断哪些方向“有问题意识但证据不足”。例如某个分支在主题中很重要，但只有间接论文支持，或者缺少关键概念、近年论文、实验评测论文，就会被标成研究空白。

更具体地说，空白主要来自三类信号：第一，taxonomy 覆盖缺口；第二，核心论文和扩展证据之间的覆盖不均衡；第三，Auditor 对证据池和关系图的审计结果。所以它不是纯文本生成，而是“结构覆盖检查 + 证据强度判断 + LLM 表达整理”的结果。

Q47：研究建议是怎么得来的？

推荐回答：

研究建议是在研究空白和可引用论文证据之上生成的。系统不会直接让模型自由发散，而是先构建 evidence snapshot，里面包括核心论文、taxonomy 分支、研究空白、证据等级和审计报告；然后 IdeaGenerationService 会基于这些结构化输入生成候选建议；最后系统会做结论准入，过滤掉没有论文支撑或与主题关系太弱的建议。

所以如果某次运行没有生成研究建议，不一定是系统坏了，也可能是证据准入策略比较保守：当前证据不足以支撑“可执行建议”，系统选择不乱编。答辩时可以说，这是我们为了可信性做的设计取舍。后续可以优化为在证据不足时给出“待验证建议”，并明确标注风险边界。

Q48：老师如果问“用户为什么要相信你们的研究建议”，怎么回答？

推荐回答：

我们不要求用户盲信建议，而是把建议和证据绑定。每条建议尽量对应研究空白、核心论文或 taxonomy 分支；前端可以查看论文证据、来源、是否用户上传、引用信息和证据等级。也就是说，建议不是最终结论，而是“基于当前证据池得到的下一步研究线索”。用户可以顺着证据池继续读论文、验证或追问。

一句话概括：我们让 AI 给建议，但不让 AI 脱离论文证据自由发挥。

Q49：你们的 RAG 是基于什么实现的？

推荐回答：

我们实现的是轻量级本地 RAG。上传的 PDF、手动导入的知识和论文候选会被保存为 KnowledgeDocument；系统会用 SimpleChunker 把文档切成带 overlap 的文本片段，再用 HashEmbedder 生成确定性的向量表示，并把 chunk 和向量存入 SQLiteVectorStore。检索时会结合向量相似度和关键词得分，形成 hybrid search。

同时，RAG 不是全局乱用。系统有 knowledge_scope 控制，可以选择不使用知识、只用当前研究资料，或者使用当前研究资料加全局共享知识。每条命中还会带上 scope、source_type、score、matched_chunk_count 和 supporting_snippets，用于前端展示“命中了什么知识，可信程度大概如何”。

Q50：你们的 RAG 和生产级 RAG 有什么差距？

推荐回答：

我们已经实现了文档导入、chunking、混合检索、作用域控制和证据展示，但还不是生产级 RAG。差距主要有三点：第一，embedding 目前是确定性 HashEmbedder，不是高质量语义 embedding；第二，rerank 还比较轻量，不如专门的 cross-encoder 或商业 reranker；第三，全文 claim verification 还没有做到每篇论文都逐条事实核验。

不过对于课程项目和本地演示，它的价值在于把 RAG 接入了完整科研调研闭环，而不是只做一个“文档问答 demo”。

Q51：你们有没有考虑 token 消耗问题？

推荐回答：

有考虑，但我们做的是工程层面的轻量控制，不是商业级成本系统。主要策略包括：第一，不把整篇 PDF 直接塞给模型，而是先 chunk 检索，只注入相关片段；第二，限制每次上下文命中的 top_k 和每个文档片段长度；第三，不同运行模式控制查询数量、检索规模和生成深度；第四，对研究总览和简报做压缩，避免多轮追问时上下文无限膨胀；第五，证据不足时会降级或抑制建议，避免为了生成完整内容而消耗更多 token。

如果老师继续追问，可以补一句：后续如果产品化上线，可以加入 token budget、缓存、增量摘要、embedding 缓存和按需全文验证，把成本控制做成显式配置。

Q52：系统里实现了哪些工具？

推荐回答：

从用户能感知和后端实际能力看，主要有这些工具：

1. **ArXiv Search**：从 arXiv 检索论文，支持主题检索、综述检索和召回补救。
2. **Semantic Scholar**：补充论文元数据、引用数、参考文献和跨论文关系。
3. **Paper Resolver**：把标题、DOI、arXiv ID 或 URL 解析成标准论文身份。

4. **Open Full-text Fetcher**: 尝试获取开放论文 PDF, 并抽取正文用于证据验证。
5. **PDF Importer**: 解析用户上传 PDF, 抽取标题、页级正文、metadata, 并写入知识库和论文池。
6. **Taxonomy Builder**: 基于论文摘要、标题和证据片段生成研究方向分类。
7. **Relationship/Evolution Builder**: 推断论文之间的引用、改进、互补、评测范围扩展和主题关联。
8. **Auditor/Corrector**: 不是外部工具, 但承担质量审计和补搜决策功能, 是 agent 闭环的重要内部工具。

可以强调: 我们把这些工具编排进同一个研究工作流, 而不是孤立调用。

Q53: 有哪些点还可以优化?

推荐回答:

可以从产品、算法和工程三个层面回答。

产品层面: 全局 PDF 上传入口还可以做得更明显; 研究总览可以进一步强化“结论如何随追问演化”; 论文详情可以更像精读助手, 给出方法、实验、局限和可复现性摘要。

算法层面: RAG 可以接入更好的 embedding 和 reranker; 全文级 claim verification 可以进一步增强; 演进图可以更多依赖真实引用和正文证据; 查询分解可以进一步用 LLM 做更稳的结构化规划。

工程层面: 可以引入更完整的异步任务队列、缓存和 token budget; 多源检索可以做更系统的失败重试和限流处理; 前端可以继续优化等待状态和可视化图布局。

Q54: 如果老师问“你们这个系统最核心的闭环是什么”, 怎么回答?

推荐回答:

最核心的闭环是: 用户提出研究问题, 系统把问题转成检索计划; Searcher 构建论文证据池; Taxonomy 和 Evolution 把论文整理成研究结构; Auditor 和 Corrector 检查证据是否足够并决定是否补搜; Synthesizer 生成研究简报、空白和建议; 用户再基于这些结果自然语言追问, 系统继承上下文继续推进。这个闭环体现了产品价值: 不是一次性回答, 而是持续组织和推进研究。

Q55: 如果老师问“你们项目最大创新点是什么”, 怎么回答?

推荐回答:

我们的创新点不在于某一个单独算法, 而在于把科研调研中分散的动作产品化成一个可复盘闭环。它把论文检索、用户上传 PDF、RAG 知识命中、证据池、taxonomy、演进图、多轮追问和 working memory 放在同一个工作台里, 让用户能围绕一个研究主题持续积累证据和结论。

如果压缩成一句话: 我们做的是 evidence-first research workspace, 而不是普通聊天机器人。

十四、候选论文池、筛选和研究图谱

Q56：候选论文池是怎么形成的？

推荐回答：

候选论文池可以理解为系统在当前研究中“认为有资格进入后续分析的全部论文集合”。它不是只来自单一搜索结果，而是由多路来源合并而成：用户上传 PDF、当前研究已有论文、arXiv 检索、Semantic Scholar 检索、综述论文检索、补搜或 facet rescue，以及必要时的背景 fallback 论文。

系统在 Searcher 阶段会先根据 Planner 生成的 retrieval plan 执行多条查询；每条查询会拿到 raw results；然后经过年份、约束、主题相关性等过滤；通过过滤的论文会进入 candidate_pool，再合并成 evidence_pool。evidence_pool 是“本轮合格证据池”，后面 taxonomy、演进图、研究空白和建议都基于它。

Q57：候选论文池和核心论文有什么区别？

推荐回答：

候选论文池更大，负责保留“可能有用的证据”；核心论文更小，负责参与深度分析和前端重点展示。系统会把所有合格论文先放进 evidence_pool，再从中挑出 analysis_papers，也就是核心分析论文。筛选核心论文时会综合主题相关性、直接证据等级、是否用户上传、引用数、年份、分类覆盖和新旧轮次等因素。

所以答辩时可以这样解释：候选池解决“查得到”，核心论文解决“看得准、读得动”。如果所有论文都直接拿来生成结论，结果会很乱；如果只保留几篇核心论文，又容易丢掉补充证据。因此我们在产品上区分核心分析和扩展证据池。

Q58：系统怎么筛查论文，避免跑题论文进入结果？

推荐回答：

论文筛查有几层。第一层是 retrieval plan 约束，比如时间范围、主题范围和用户追问里的重点。第二层是相关性评估，系统会对论文标题、摘要、关键词和分类做 topic gate 和 facet gate，区分 direct、adjacent 和 candidate。candidate 级别的低相关论文会被过滤。第三层是去重和合并，避免同一篇论文通过 arXiv 和 Semantic Scholar 重复出现。第四层是 evidence admission，只有足够相关的论文才会进入核心分析或强结论。

如果某些边缘论文仍然出现，系统会尽量把它们放在扩展证据或背景证据里，而不是让它们主导研究建议。这个策略对应我们之前说的“不要用无关论文硬填结果”。

Q59：如果外部检索结果很差，系统怎么处理？

推荐回答：

系统不会立刻失败，而是会尝试几种补救：扩展查询、OR 召回、Semantic Scholar 补充、facet rescue、综述论文检索，以及在严格约束没有命中时保留背景 fallback 论文。但 fallback 论文会被标成 background，不应该作为强证据。也就是说，系统尽量保证流程不中断，但会通过证据等级告诉用户哪些结果可靠、哪些只是背景参考。

Q60: taxonomy 是怎么来的?

推荐回答:

taxonomy 是基于当前论文证据池构建的研究方向分类。系统会把论文标题、摘要、关键词、分类信息、用户上传 PDF 中可用片段, 以及可能的综述文本整理成 evidence brief, 然后交给 Taxonomy Builder 生成候选分支。每个分支包含名称、描述和 required concepts。

生成后, 系统还会把论文重新分配到 taxonomy 分支上。分配不是只看论文原始分类, 比如 cs.CL 或 cs.CV, 而是根据分支名称、描述、required concepts 与论文标题、摘要、关键词之间的匹配分数来判断。最后每个分支会有 paper_count、coverage_score、evidence_tier 等指标, 用来判断这个方向证据强不强。

Q61: taxonomy 里的分支可信度怎么理解?

推荐回答:

taxonomy 分支不是“标准学科分类”, 而是系统根据当前证据池生成的调研结构。它的可信度取决于该分支下有多少论文支持、关键概念是否覆盖、是否有研究空白与它对应。如果某个分支论文数量多、coverage score 高, 就更适合作为主线; 如果论文少但 gap 多, 就更适合作为潜在研究空白。

答辩时可以强调: taxonomy 的作用是帮助用户快速建立领域地图, 而不是替代专家最终定义学科分类。

Q62: 演进图是怎么来的?

推荐回答:

演进图是基于论文关系生成的候选研究关系图。系统会先找强证据关系, 比如 Semantic Scholar 返回的引用关系; 如果 A 明确引用 B, 就可以生成 citation 边。然后系统会根据标题、摘要、taxonomy 分支、关键词和方法描述推断一些弱关系, 比如 complements、scope extension、related 等。最后还会进行证据验证和稀疏化, 避免所有论文两两相连形成“毛线团”。

所以演进图不是简单把所有论文连起来, 而是先保留明确引用, 再补充少量高置信的主题关系, 并限制弱边数量。前端展示关系类型和置信度, 就是为了提醒用户这是研究线索, 不是绝对因果链。

Q63: 演进图里的边类型分别代表什么?

推荐回答:

可以这样解释:

1. **citation**: 明确引用关系, 证据最强, 表示目标论文引用了源论文。
2. **improves / addresses**: 后续论文针对前人方法、限制或问题做改进或解决。
3. **complements**: 两篇论文从不同角度补充同一研究路线。
4. **expands scope / scope_extension**: 后续论文扩展了应用场景、评测范围或任务范围。

5. **related / theme association**：主题相关或 taxonomy 分支相近，只能作为候选线索，不能解释成严格演进。

如果老师追问可靠性，可以补一句：我们在前端和数据结构中区分 confirmed、inferred、candidate 等证据等级，避免把所有关系混成同等可信。

Q64：为什么演进图有时边不多？

推荐回答：

这是有意的保守设计。早期版本如果按关键词相似度大量连边，图会看起来很热闹，但容易误导用户，以为每条边都有强学术关系。现在我们更倾向于少而可信：明确引用边优先，弱相关边要经过置信度和数量限制。边少不一定是坏事，说明系统没有把弱关系包装成强结论。

答辩时可以把这个解释成可信性设计：科研助手不是为了画满图，而是为了让图上的关系有解释价值。

十五、代码与实现细节追问

Q65：为什么用 SQLite？数据库里主要存了什么？

推荐回答：

我们选择 SQLite 是因为课程项目以本地部署和演示稳定为主，SQLite 不需要额外服务，便于组员复现。数据库里不只存聊天记录，还存了会话、消息、研究任务、任务运行事件、工作台快照、working memory、知识文档、vector chunks、研究论文池、skill 配置等。这样每次研究不是临时生成完就丢，而是可以回看、合并和复盘。

如果后续产品化上线，可以把 SQLite 替换成 PostgreSQL 或带向量索引的数据库，但服务层已经做了 repository 抽象，迁移路径比较清楚。

Q66：前端怎么知道 Agent 跑到哪一步？

推荐回答：

后端在研究任务执行过程中会记录 task events。每个事件包含 stage、status、message、payload 和 sequence。Agent 节点执行前后会把当前状态写成事件，例如 Planner、Searcher、Taxonomy、Evolution、Auditor、Corrector、Synthesizer 的动作、论文数量、边数量、分支数量等。前端读取这些事件后，就能渲染运行轨迹和 Agent node 状态图。

这个设计的好处是用户等待时不是盯着空白页面，而是能看到系统正在规划、检索、审计还是生成结论。

Q67：为什么系统有时候不给研究建议？

推荐回答：

这是结论准入策略在起作用。系统会先判断当前证据池是否足以支持建议，如果直接论文太少、

证据等级太弱，或者建议无法绑定到研究空白和核心论文，就会抑制研究建议。这样看起来比“永远生成三条建议”更保守，但能降低幻觉风险。

答辩时可以说：我们宁愿让系统承认证据不足，也不希望它把没有论文支撑的灵感包装成确定研究结论。

Q68：PDF 上传后是不是把全文都喂给大模型？

推荐回答：

不是。PDF 上传后会先解析为页级文本，再切成 chunk 建索引。真正生成时，系统只根据研究主题和追问召回相关片段，而不是把整篇 PDF 全部放进 prompt。这样可以控制 token 消耗，也可以减少无关正文对结果的干扰。

如果用户问某篇论文的细节，系统可以优先使用该论文的相关片段；如果只是生成研究总览，就只取对主题有帮助的证据。

Q69：你们的向量检索用的是什么 embedding？

推荐回答：

目前为了本地可复现和低依赖，我们使用的是确定性的 HashEmbedder，并结合关键词得分做 hybrid search。它不是生产级语义 embedding，但好处是无需外部 embedding API、运行稳定、测试可复现。后续优化时可以直接替换为更强的 embedding 模型和 reranker，例如 bge、e5、OpenAI embedding 或 cross-encoder reranker。

这也是我们项目的一个工程取舍：课程项目阶段优先保证闭环和可解释，生产化阶段再增强语义检索质量。

Q70：外部 API 限流或失败怎么办？

推荐回答：

Searcher 做了降级和补救。比如 Semantic Scholar 限流时，系统不会直接崩掉，而是继续使用 arXiv、已有论文池、召回补救或 facet rescue。检索结果不足时，系统也会在 retrieval_outcome 中记录证据受限状态，提醒用户当前结果不适合强结论。

所以系统不是假设外部工具永远可用，而是把失败当作正常工程场景处理。

Q71：为什么要区分 evidence_pool 和 analysis_papers？

推荐回答：

evidence_pool 是所有通过过滤的合格证据，负责保证覆盖；analysis_papers 是从 evidence_pool 里挑出的核心分析论文，负责保证深度。如果直接把几十篇候选论文都交给后续生成，结果会变得混乱，也会增加 token 消耗；如果只保留几篇核心论文，又容易丢掉补充线索。因此我们用 evidence_pool 保留证据广度，用 analysis_papers 控制分析深度。

一句话：证据池负责“查得全”，核心论文负责“讲得清”。

Q72：用户上传论文为什么有时没有进入核心论文？

推荐回答：

用户上传论文会被提高权重，但不会无条件进入核心论文。原因是系统仍然要判断它和当前研究主题是否相关。如果用户上传论文与主题偏离，强行放进核心论文会污染结论。我们的策略是：用户上传论文优先被纳入当前研究资料和候选证据池，如果它与主题相关，就更容易进入核心分析；如果相关性弱，就可能作为扩展证据或上下文参考。

Q73：研究总览和本次结果有什么区别？

推荐回答：

本次结果是某一轮研究任务的局部结果，适合看这轮追问带来了什么变化；研究总览是同一会话下多轮任务的合并视图，适合看整个研究主题的累计进展。比如首次研究建立大方向，后续追问补充近三年论文或聚焦某个分支，总览会把这些结果融合起来。

Q74：删除资料为什么可能涉及 foreign key？

推荐回答：

因为知识文档、vector chunks、研究论文池和工作台结果之间存在引用关系。数据库用外键保证数据完整性，避免出现“论文表还引用一个已经不存在的知识文档”的脏数据。删除资料时需要按顺序清理关联记录，或者做级联删除。这也是我们在工程上需要继续打磨的地方。

Q75：你们怎么测试系统不是只能跑 demo case？

推荐回答：

我们做了两类验证。第一类是自动化测试，覆盖查询分解、检索补救、论文相关性过滤、知识召回等关键逻辑；第二类是手动主链测试，用不同主题跑新建研究、PDF 导入、追问、研究总览和前端展示，记录 bad case 并迭代修复。我们之前发现过追问不补搜、引用数丢失、演进图边太弱、共享知识污染等问题，都是通过主链手测暴露出来的。

Q76：为什么前端要分 Overview / Evidence / Map / Context / Run？

推荐回答：

科研调研结果信息密度高，如果只用聊天气泡展示，用户很难同时看到结论、论文、证据、图谱和系统运行过程。我们在工作台拆成五个区域：Overview 看研究简报和下一步，Evidence 看论文证据，Map 看 taxonomy 和关系图，Context 看知识命中，Run 看 Agent 执行轨迹。这个拆分是为了让用户更像在使用研究工作台，而不是普通聊天窗口。

Q77：Skill 功能做到什么程度？

推荐回答：

目前系统有 skill 注册和管理的基础，但还不是完整的可执行插件生态。也就是说，它更像预留扩展点，用来描述某类能力或工具组合；现阶段主链核心价值仍然来自论文检索、知识导入、

RAG、Agent workflow 和工作台展示。答辩时不建议把 skill 作为主要卖点，可以作为后续扩展方向。

Q78：系统最大的工程风险是什么？

推荐回答：

主要风险有四类：第一，外部论文检索质量不稳定；第二，LLM 生成可能受证据不足影响；第三，PDF 解析质量取决于论文格式和是否可抽取文本；第四，证据准入太宽会幻觉，太严又可能不给建议。我们通过多源检索、相关性过滤、补搜、降级、证据展示和运行轨迹来降低这些风险。

Q79：如果继续开发，最值得优化哪些代码和能力？

推荐回答：

最值得优先做五件事：第一，替换更强 embedding 和 reranker，提高 RAG 命中质量；第二，增强全文级 claim verification，让论文详情更像真正的精读助手；第三，补全全局 PDF 上传入口；第四，引入更完整的异步任务队列和缓存，提升并发和稳定性；第五，进一步优化演进图关系验证，让图谱既好看又更可信。